

Evaluating MTL Mode Parameters in Real-world Scenarios

Mike Almeloo

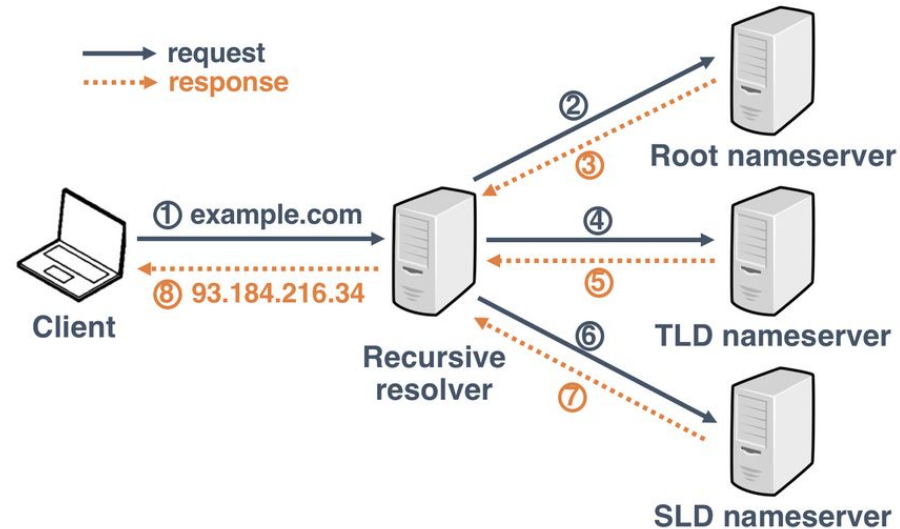
Supervised by:

Roland van Rijswijk-Deij

Ginevra Fabrizio

Introduction: *the Domain Name System (DNS)*

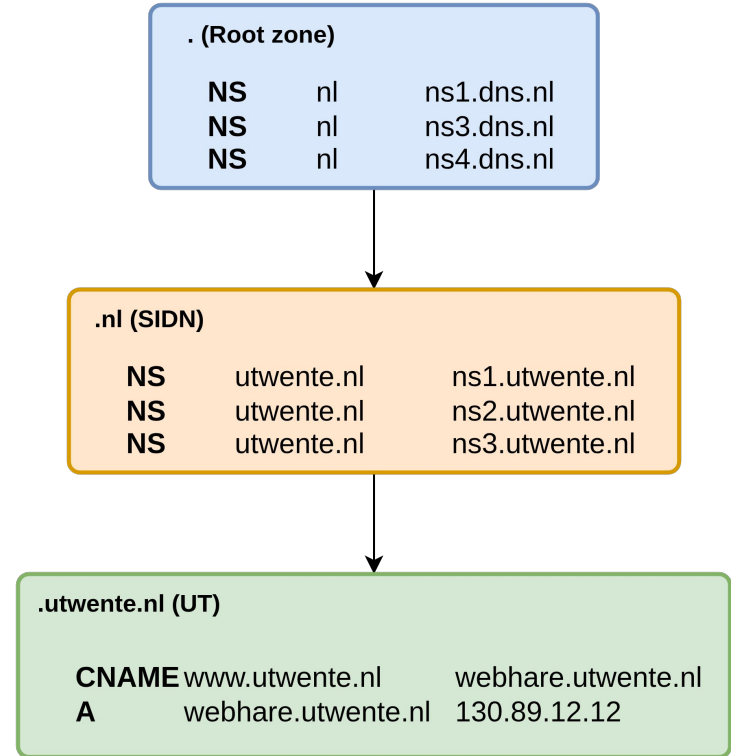
- Computers need IP addresses in order to communicate
 - Example: 127.0.0.1
- **Q:** How does the computer know how to reach www.utwente.nl?
- The DNS is *distributed* and *recursive*
 - No central domain name → IP database
 - Start resolving at the root zone



Source: <https://doi.org/10.1145/3340301.3341122>

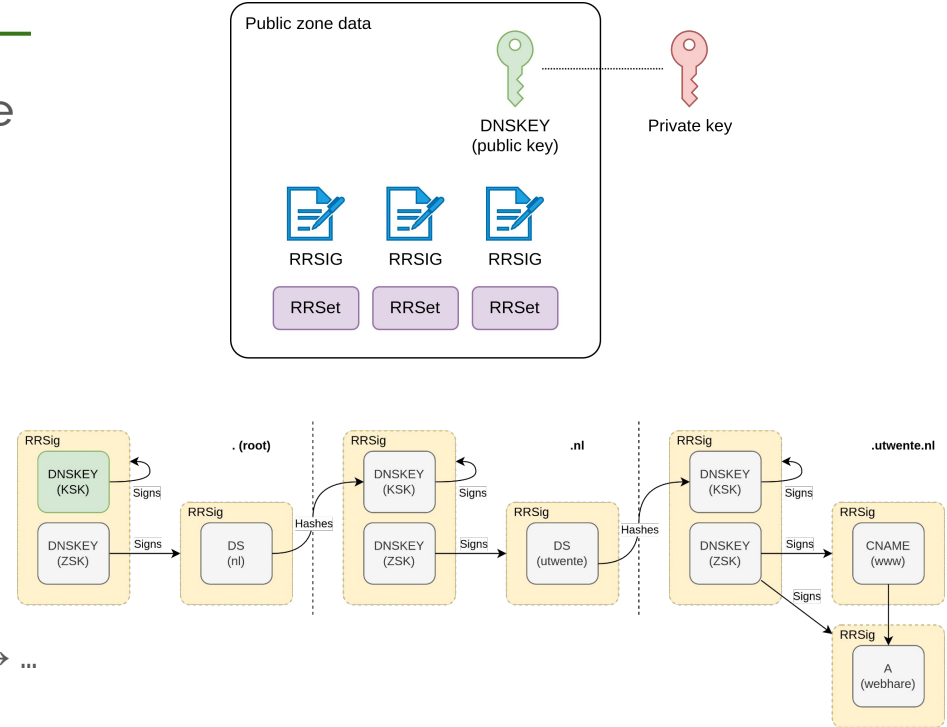
Introduction: *the DNS*

- The DNS is structured into records
 - Records have a *type*, *name* and *data*
 - Each record type has a different meaning
- Transport is usually UDP
 - Effective response size limit of 1,232 bytes (to avoid fragmentation)



Introduction: *DNSSEC*

- **Problem:** the DNS is very insecure by default
 - How do we know whether we are contacting the right server?
- **Solution:** let's cryptographically sign all records!
- Creates a chain of trust
 - Root zone → TLD zone → SLD zone → ...



Introduction: *Post-Quantum DNSSEC*

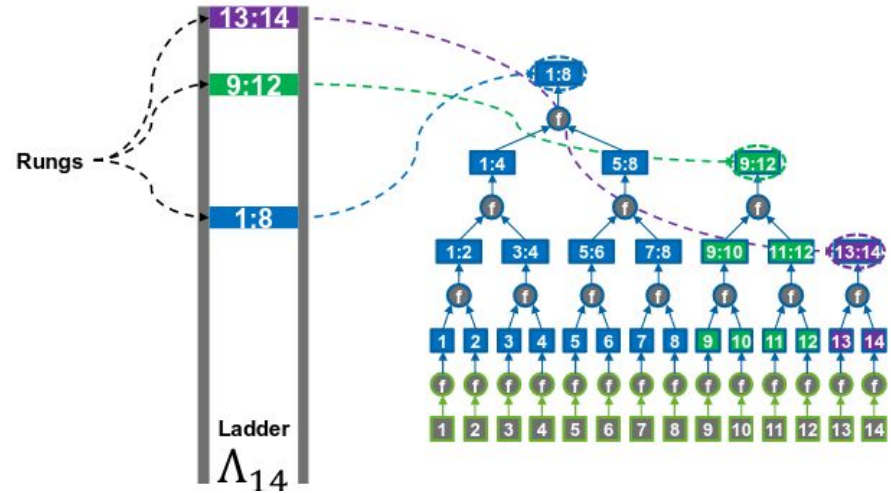
- **Problem:** Quantum computers can break DNSSEC in polynomial time
 - Current algorithms use either RSA or ECC
- We could substitute post-quantum algorithms, but they either:
 - produce large signatures;
 - are prohibitively slow.
- Remember: 1,232 byte limit!
 - So what do we do?

Family	Name variant	A	Sizes (bytes)		CPU time (lower is better)	
			Public key	Signature	Signing	Verification
Elliptic curves	Ed25519	✗	32	64	0.15	1.3
Factoring	RSA 2048	✗	272	256	80	0.4
Lattices	ML-DSA 44	✓	1,312	2,420	1 (baseline)	1 (baseline)
Symmetric	SLH-DSA 128s	✓	32	7,856	14,000	40
	SLH-DSA 128f	✓	32	17,088	720	110
	SLH-DSA 128-24	✗	32	3,856	7,000,000	4
	LMS M24_H20_W8	✓	48	1,112	2.9	8.4
Lattices	FN-DSA 512	✗	897	666	3	0.7
Lattices	HAWK 512	☹	1,024	555	0.25	1.2
Proof of knowledge	MQOM L1-gf16-fast-5r	☹	60	3,280	8	20
	SDiTH SDiTH2-L1-gf2-fast	☹	70	4,484	15	40
	FAEST EM-128f	☹	32	5,060	4.2	9
Isogeny	SQlsign I	☹	65	148	300	50
Multivariate	MAYO one	☹	1,420	454	2.1	0.4
	MAYO two	☹	4,912	186	1.1	0.8
	QR-UOV I-(127 156 54 3)	☹	24,225	200	9.3	20
	SNOVA (24,5,4)	☹	1,016	248	1.2	1.7
	SNOVA (25,8,3)	☹	2,320	165	1	1.5
	SNOVA (37,17,2)	☹	9,842	124	0.8	1.3
	UOV Is-pkc	☹	66,576	96	0.3	2.4
	UOV Ip-pkc	☹	43,576	128	0.3	2

<https://blog.cloudflare.com/ml-dsa-will-have-to-do/>

Introduction: *MTL Mode*

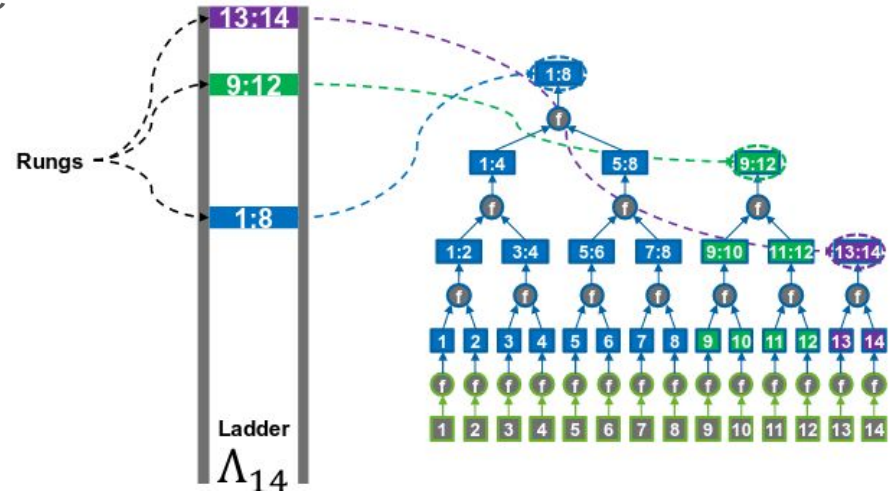
- We need a PQ algorithm *now*
 - ...but none of the options are very good
- Idea: what if we sign multiple records with 1 PQ signature?
 - Then we only occasionally need to validate an expensive signature!



Source: https://doi.org/10.1007/978-3-031-30872-7_16

Introduction: *MTL Mode*

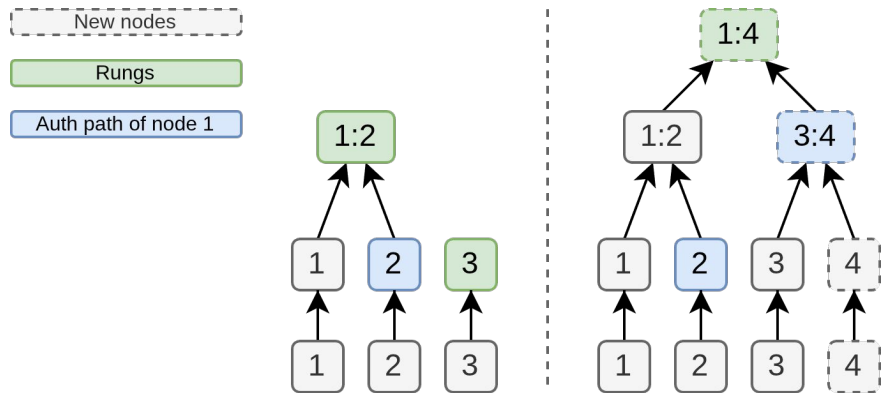
- RRSets covered by multiple Merkle trees (ladder)
- Two types of RRSIGs:
 - **Full** (Rungs + PQ sig + Merkle proof)
 - **Condensed** (Merkle proof)
- Backwards compatibility
 - Full sig can validate all condensed sigs created up until that point
 - (given that they originate from the same ladder)



Source: https://doi.org/10.1007/978-3-031-30872-7_16

Introduction: *MTL Mode*

- Zone modifications → extend the ladder
- New authentication paths will pass through old rungs
 - Backwards compatibility!
- Note: ladders only grow, never shrink
 - They need to be recreated at some point



Motivation

- MTL Mode promises many things for PQ-DNSSEC
 - Reduced (amortized) signature sizes, verification cost, latency, ...
- ...but how well does this really work in practice?
 - Preferably, we want to validate full (PQ) signatures as infrequently as possible
 - Largely determines validation cost, latency, avg sig size, ...
 - Useful for adversarial analysis
- More specifically: **How do various system parameters influence the time between full signature fetches in an MTL-enabled DNSSEC deployment?**
 - RRSIG lifetime, record update rate, query rate, MTL strategy, ...

Methodology

1. Build a **probabilistic model** to quickly evaluate parameter changes
2. Build a **simulator** for accuracy + more complex scenarios
3. **Evaluate** both against a real-world zone + resolver
 - a. Zone data: **.nl** (*SIDN*)
 - b. Resolver data: **SURF** (*Dutch research + education org*)

Model

- Three events cause full sig invalidation:
 - Underlying record update (**external update**)
 - Re-signing due to imminent expiry (**periodic update**)
 - Merkle tree **ladder recreation**
- Let's assume:
 - These events are independent;
 - Resolver queries are independent;
 - Resolver queries uniformly.
- Worst-case assumptions!

Model

Zone behaviour

Single-record invalidation probability

$$p(t) = 1 - e^{-\mu_{record}t} \cdot \left(1 - \frac{t}{T_S}\right) \cdot l(t)$$

Resolver 'view'

Cumulative probability over ladder

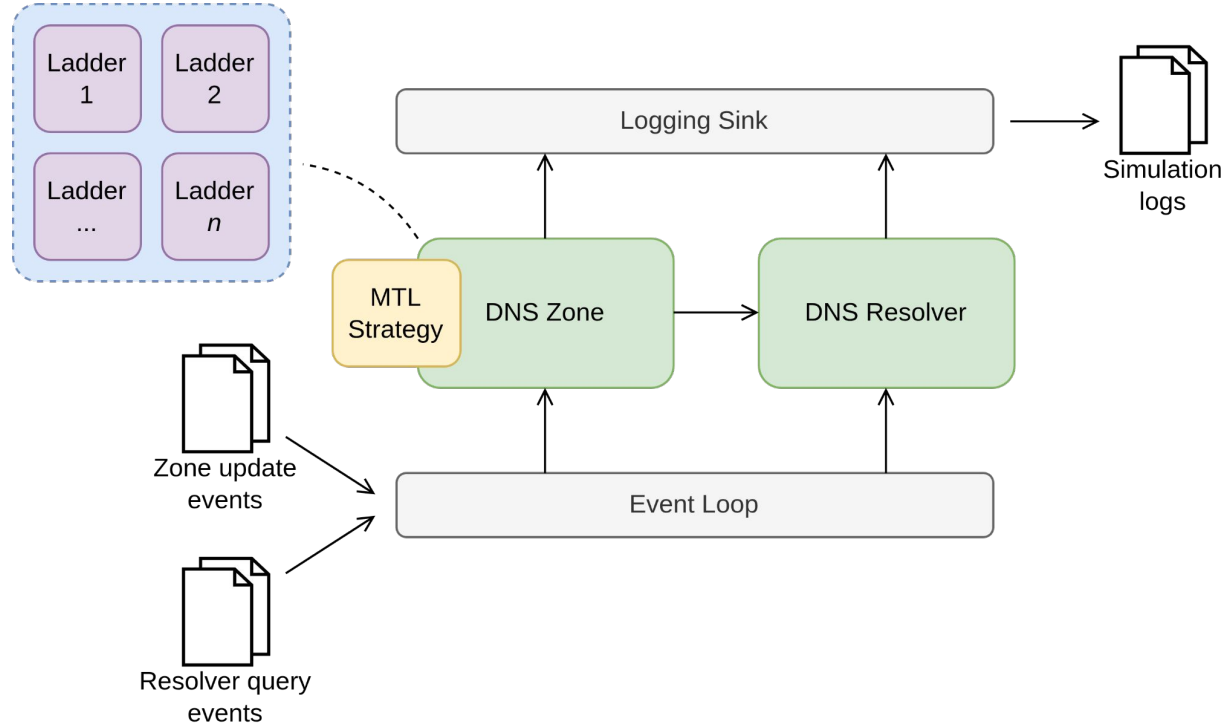
$$P(T > t) = \exp \left(-\lambda \int_0^t p(s) ds \right)$$

Exp. inter-sig time
(endurance)

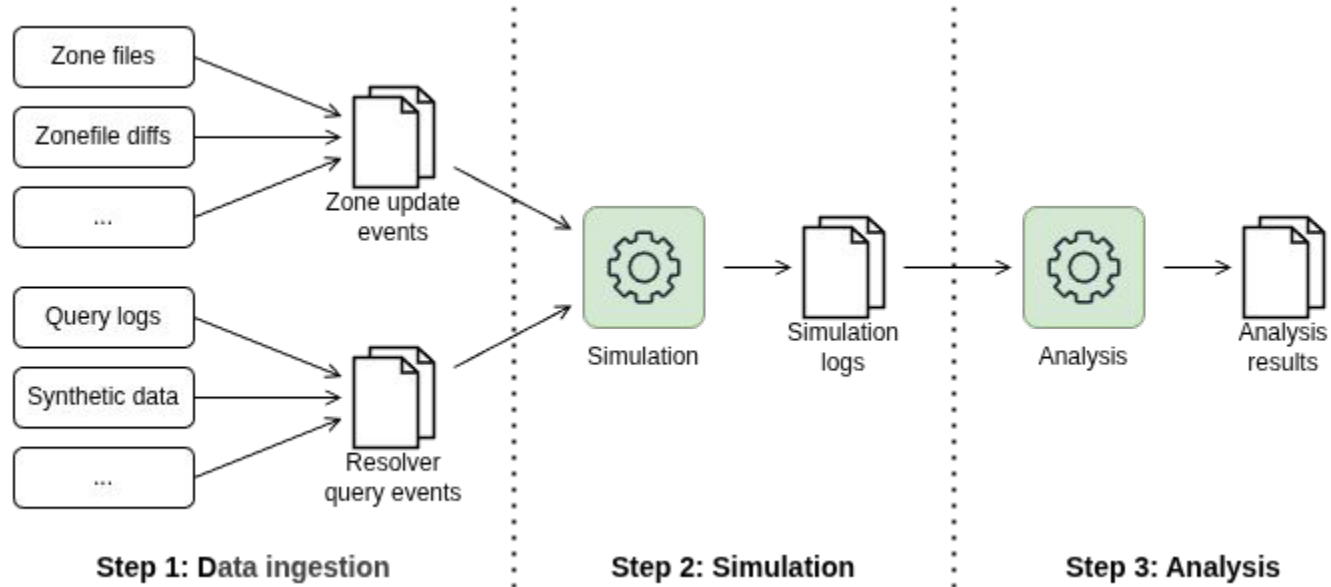
Expected endurance

$$E[T] = \int_0^\infty P(T > t) dt$$

Simulator - *Architecture*

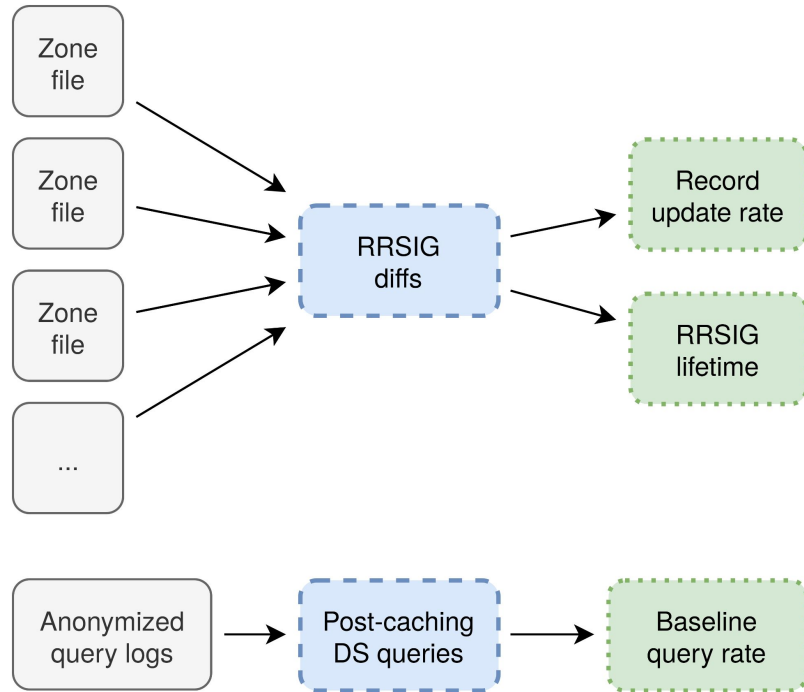


Simulator - *Workflow*

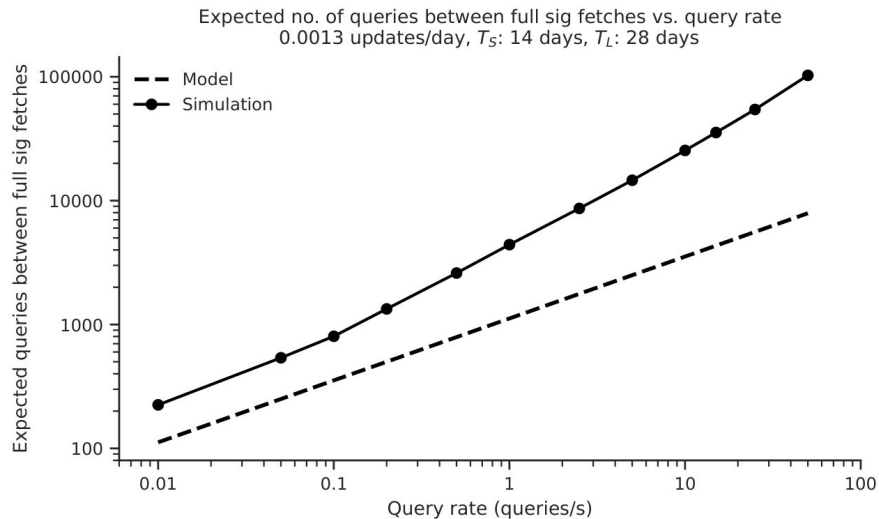
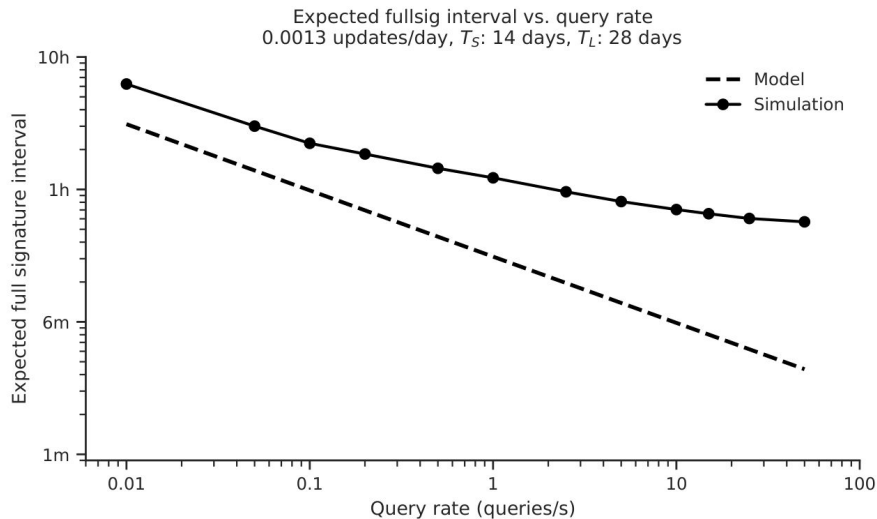


Evaluation: *Data collection*

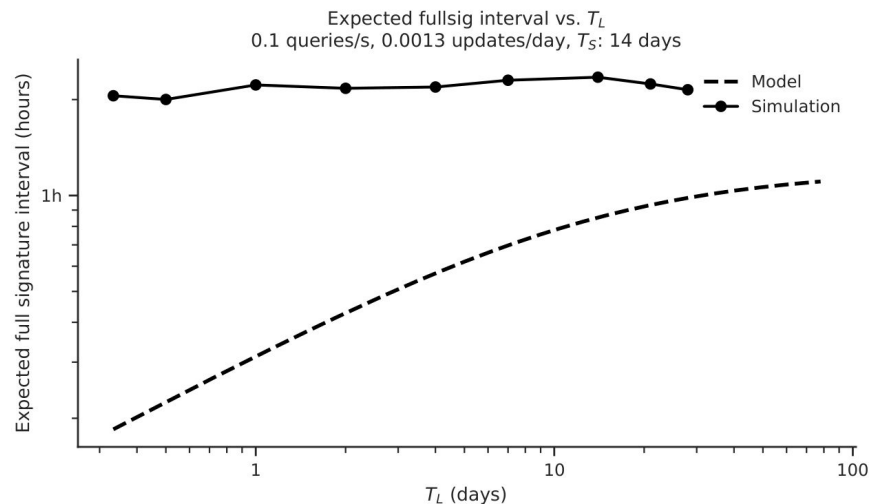
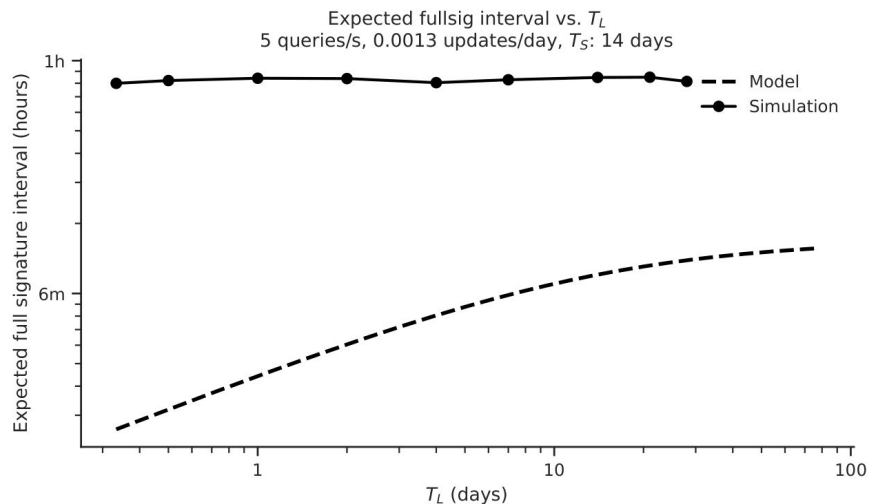
- ~28 days of .nl zone files
- 24 hours of anonymized client-side query logs



Evaluation: *Varying query rates*



Evaluation: *Varying max. ladder lifetime*



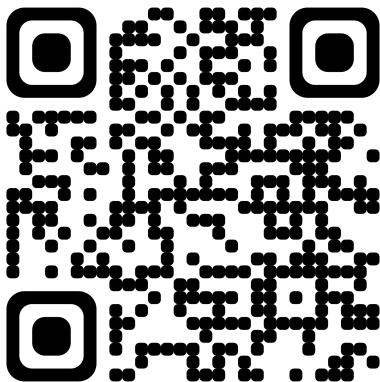
Conclusions

- Signature endurance is heavily influenced by:
 - Query rate (resolver param)
 - Publishing interval (zone param)
- Low ladder lifetime appears to be fine in practice
 - Reduces adversarial opportunities
- Model paints a worst-case picture
 - May be useful for zones and resolvers to plan for resource load

Conclusions

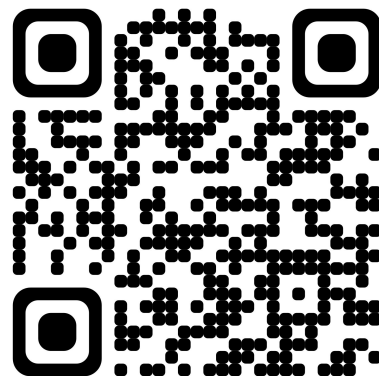
- Most important parameters are independent from MTL mode
 - Deployment flexibility for zone operators
- What does this mean for stakeholders?
 - **Resolvers:** fetch frequency is largely self-induced (query rate)
 - **Zone operators:**
 - The system appears to scale well
 - Large resolvers are predictable
 - Deployment flexibility

Paper



<https://purl.utwente.nl/essays/111423>

mtlsim



<https://github.com/malmeloo/mtlsim>

Future works

- Move sig to new ladder on re-sign
 - Eliminates ladder recreation events
- Model update (implement batching interval)
- More deployment strategies
 - E.g. 1 ladder per signature type (DS, NSEC3, ...)
- Full signature pre-fetching
- Test on different zones!